DATA 612 · Statistical Programming in R · Week 8

8.3 dbplyr: Databases

tbl() is lazy SQL. collect() brings a result into R. Use an in-memory SQLite.

1. When a data frame is not enough

A tibble lives in memory. That is fine for `flights`. It is not fine for a table that is larger than RAM, or that several people query at once, or that already lives in SQL.

A **database** stores the tables on disk and runs the filter **there**. You pull back only the rows you need.

`dbplyr` translates dplyr verbs into SQL. It does **not** load with `library(tidyverse)`. Install `dbplyr`, `DBI`, and a backend (`RSQLite` is enough for this course).

We will use an in-memory SQLite copy of `flights`. Skip large local `.sqlite` files; `":memory:"` is enough to see the translation.

---

2. Connect, copy, `tbl()`

```
library(tidyverse)
library(DBI)
library(RSQLite)
library(dbplyr)
library(nycflights13)

con <- dbConnect(SQLite(), ":memory:")
copy_to(con, nycflights13::flights, "flights", temporary = FALSE)
flights_db <- tbl(con, "flights")
```

`":memory:"` is a database that exists only in this R session. `copy_to()` writes a local data frame into that database. `tbl(con, "flights")` is a **lazy** handle: it looks like a tibble when you print it, but the rows are still in SQLite.

`dbDisconnect(con)` when you are done.

---

3. Lazy verbs, then `collect()`

dplyr verbs on `flights_db` do not run immediately. They build a query. Printing shows a preview (often 10 rows). `show_query()` prints the SQL. `collect()` pulls the result into a local tibble.

```
flights_db |>
  filter(month == 1) |>
  select(year:dep_delay)

flights_db |>
  filter(month == 1) |>
  show_query()

jan <- flights_db |>
  filter(month == 1) |>
  collect()
```

`jan` is an ordinary tibble. Everything after `collect()` is local. Printing `flights_db` is only a preview; it is not the full table in memory.

Not every tidyverse verb is SQL. `pivot_longer()`, `separate()`, and most tidyr helpers need a local frame. `filter()`, `select()`, `mutate()`, `arrange()`, `group_by()`, `summarize()`, and the joins usually translate.

```
flights_db |>
  group_by(carrier) |>
  summarize(mean_del = mean(dep_delay, na.rm = TRUE), n = n())
```

That summary still lives in the database until you `collect()`. ggplot2 needs a local data frame, so plot only after `collect()`.

---

4. Close the connection

```
dbDisconnect(con)
```

If you skip this, R will close it when the session ends. Close it yourself so you do not leave a lock on a file-backed database later.

---

5. Practice

1. Copy `flights` into an in-memory SQLite database. Mean `dep_delay` by `carrier` from the remote table. `collect()`, then plot.
2. On the same remote table, `filter(month == 1)` and `show_query()`. What SQL did dbplyr write?